

Quantum-Inspired Optimization for Entanglement Routing in Quantum Networks: A Hamiltonian and Tensor-Network Approach

Arnav Kapoor
QiOpt4QNet Project
arnav@example.com

August 5, 2026

Abstract

We formulate entanglement-routing in quantum repeater networks as a quadratic unconstrained binary optimization (QUBO) problem and compare three solver families—Metropolis annealer, tensor-network (boundary-MPS) compressor, and OpenJij simulated annealing—on a standardized benchmark suite of chain and grid topologies. The QUBO encodes fidelity, latency, success probability, and capacity constraints into a single energy functional whose minima correspond to high-utility bundle-selection configurations. We extend this formulation with three physically motivated Hamiltonian terms—soft congestion, memory decoherence risk, and a direct (slack-free) energy encoding—and introduce a streaming/adaptive variant that incrementally updates solutions as requests arrive. On instances up to 24 requests, both the Metropolis annealer and the tensor-network compressor find the same capacity-limited served-ratio frontier (100% at the lightest loads down to 37–46% at maximal contention), with utilities agreeing to within $\sim 5\%$; the tensor-network solver is 6–370 $\times$ faster. Bond dimension has negligible effect even down to $\chi = 1$, indicating that local coupling dominates. Streaming matches or exceeds batched solution quality while reducing wall-clock time by 60–120 $\times$. We then close the four future directions of the previous version: a time-dependent decoherence-aware router that lifts delivered fidelity, a hard-constraint MPS solver that eliminates the feasibility-repair step, a Q-learning adaptive router that matches annealer quality with 25 $\times$ cheaper online inference, and hardware parameter profiles showing a 2.8 $\times$ delivered-fidelity spread across current platform families. Our code is open-source at <https://github.com/arnavk23/QiOpt4QNet-Simulator>.

1 Introduction

Quantum networks distribute entanglement—a fundamentally quantum resource—between distant nodes, enabling applications in quantum key distribution (QKD), distributed quantum computing, and blind quantum computing [1]. The central routing problem is: given a set of concurrent entanglement requests between node pairs, select for each request a path and a purification schedule (collectively a *bundle*) that maximizes end-to-end fidelity while respecting edge and memory capacities.

This is a constrained combinatorial optimization problem whose structure resembles resource allocation in communication networks, but with physical constraints unique to quantum systems: probabilistic entanglement generation, exponential fidelity decay under decoherence, and the need for entanglement-swapping operations that consume memory qubits at intermediate repeater nodes [2, 3].

We make the following contributions:

1. We formalize entanglement routing as a QUBO/Ising energy minimization problem (Sec. 3), enabling the application of quantum-inspired solvers.
2. We implement and benchmark three solver families: a Metropolis annealer with local Hamiltonian updates, a tensor-network (boundary-MPS) compressor [4], and OpenJij simulated annealing [12].
3. We introduce physically motivated Hamiltonian extensions—soft congestion and memory decoherence risk—and compare them against the standard penalty-based QUBO encoding.
4. We propose a streaming/adaptive variant that performs incremental local updates as requests arrive, matching batched solution quality at reduced latency.
5. We close the four future directions of the previous version: a time-dependent decoherence-aware

router, a hard-constraint MPS solver, a Q-learning adaptive router, and hardware parameter profiles (Sec. 6).

6. We release an open-source benchmark suite¹ for reproducible evaluation.

Our work is directly inspired by Rosales [4], who introduced a Hamiltonian/Metropolis and boundary-MPS approach for adaptive multi-demand routing in QKD networks. We transplant this methodology to the entanglement-routing setting, where resources are probabilistic rather than deterministic, and identify where the assumptions differ.

2 Related Work

2.1 Tensor-Network Optimization

Tensor-network methods for combinatorial optimization have gained traction as classical surrogates for quantum annealing. Hao et al. [5] demonstrated constrained optimization via tensor networks on open-pit-mining instances, finding that MPS compression locates higher-quality solutions than simulated annealing on problems with strong local coupling. Preisser et al. [6] provide a systematic methodology for variational MPS optimization, including sweep protocols and truncation schedules that we adapt here. Nakada et al. [7] show how hard constraints can be encoded directly into tensor-network structure rather than as penalty terms—a technique we compare against our slack-variable QUBO baseline. Mata Ali [8] and Mata Ali et al. [9] give theoretical scaling frameworks that predict when tensor-network methods outperform classical solvers.

2.2 Quantum Network Routing

Existing approaches to entanglement routing include heuristic path selection [13], linear programming relaxations [14], and reinforcement learning [10]. The QUBO formulation is less explored; the closest prior work is the QKD routing model of Rosales [4], on which we build directly. Berezutskii et al. [11] provide a broad review connecting tensor-network methods to quantum computing applications.

3 QUBO Formulation for Entanglement Routing

We model the quantum network as a graph $G = (V, E)$ with edge capacities C_e (Bell pairs per time slot) and

node memory capacities M_v (qubit slots). A request r specifies a source-destination pair (s_r, d_r) , a minimum fidelity $F_{\min}$, and a weight w_r .

For each request r , the bundle generator produces a set of candidate bundles B_r , each consisting of a path p and a purification profile q (rounds of BBPSSW purification per link). A bundle b is characterized by:

- utility $u_{r,b}$ (reward minus cost),
- edge demands $\{d_{r,b}^e \in \mathbb{N}\}$,
- memory demands $\{m_{r,b}^v \in \mathbb{N}\}$,
- latency $\ell_{r,b}$.

The utility is defined as:

$$u_{r,b} = w_r \cdot p_{\text{succ}} \cdot (1 + \beta \cdot \max(0, F - F_{\min})) - \lambda_\ell \cdot \ell_{r,b} - \lambda_c \cdot \text{cost}_{r,b}, \quad (1)$$

where p_{succ} is the end-to-end success probability, F the end-to-end fidelity after concatenated entanglement swapping and BBPSSW purification, and cost is the Bell pair expenditure.

The full Hamiltonian is:

$$H = H_{\text{util}} + H_{\text{one-per-req}} + H_{\text{edge}} + H_{\text{mem}}, \quad (2)$$

with

$$H_{\text{util}} = - \sum_{r,b} u_{r,b} x_{r,b}, \quad (3)$$

$$H_{\text{one-per-req}} = A \sum_r \sum_{i < j} x_{r,i} x_{r,j}, \quad (4)$$

$$H_{\text{edge}} = B \sum_e (L_e + s_e - C_e)^2, \quad (5)$$

$$H_{\text{mem}} = D \sum_v (M_v + s_v - M_v^{\max})^2, \quad (6)$$

where $x_{r,b} \in \{0, 1\}$ indicates whether bundle b is selected for request r , $L_e = \sum_{r,b} d_{r,b}^e x_{r,b}$ is the edge load, s_e and s_v are slack variables, and A, B, D are penalty coefficients. This is a standard penalty-based QUBO.

3.1 Direct (Slack-Free) Hamiltonian

As an alternative, we define a physical Hamiltonian with no slack variables:

$$\begin{aligned} H_{\text{phys}} = & - \sum_{r,b} u_{r,b} x_{r,b} + \lambda_1 \sum_r (\sum_b x_{r,b} - 1)^2 \\ & + \lambda_2 \sum_e \max(0, L_e - C_e)^2 \\ & + \lambda_3 \sum_v \max(0, M_v - M_v^{\max})^2. \end{aligned} \quad (7)$$

This formulation uses fixed-scale penalty parameters λ_i that can be set once based on the utility scale rather than tuned per instance, addressing the known fragility of penalty-based QUBO [7].

¹<https://github.com/arnavk23/QiOpt4QNet-Simulator>

3.2 Soft Congestion

We add a congestion term that penalizes near-saturation of edges before the hard capacity limit is reached:

$$H_{\text{cong}} = \lambda_{\text{cong}} \sum_e \max(0, L_e/C_e - \theta)^2, \quad (8)$$

where $\theta \in [0.5, 0.9]$ is a warning threshold. This provides a smooth gradient that biases the solver toward load-balanced configurations.

3.3 Memory Decoherence Risk

Entangled pairs stored in memory undergo T1/T2 decoherence. We model this as a risk term:

$$H_{\text{risk}} = \lambda_{\text{risk}} \sum_{r,b} (1 - e^{-\ell_{r,b}/\tau_{\text{mem}}}) x_{r,b}, \quad (9)$$

where τ_{mem} is the characteristic memory coherence time and $\ell_{r,b}$ is the expected storage duration (proportional to latency). This penalizes bundles that leave qubits waiting in memory. In the time-dependent router (Sec. 6) we use a fidelity-aware variant that penalizes $1 - F_{r,b}(\text{decay}(t))$ with the full storage-fidelity law, aligning the risk term with the delivered (post-decay) fidelity metric.

4 Solvers

We implement three solver families, all consuming the same Hamiltonian and differing only in search strategy.

4.1 Metropolis Annealer

The Metropolis annealer treats a configuration $\{x_{r,b}\}$ as a physical microstate and explores the energy landscape via local bit-flip moves [4]. At each step, it proposes changing one request’s bundle selection, computes the local energy change ΔH (which is cheap since H is a sum of local and pairwise terms), and accepts with probability $\min(1, e^{-\Delta H/T})$. Temperature follows a geometric cooling schedule from $T_0 = 10$ to $T_{\text{min}} = 10^{-3}$ over up to 5000 iterations. The initial configuration is a greedy utility-density seed.

4.2 Tensor-Network (Boundary-MPS) Compressor

Instead of sampling configurations sequentially, the tensor-network solver [4] represents the joint distribution over all requests’ bundle choices as a Matrix Product State (MPS). Each request r is assigned a rank-3 tensor T_r with physical index ranging over $B_r \cup \{\text{reject}\}$

and virtual bond indices carrying correlations to neighboring requests (ordered by edge/memory adjacency). The MPS is contracted via left-to-right and right-to-left sweeps, truncating to bond dimension χ via SVD after each contraction. The dominant bundle choice per request is decoded from the final MPS, and any residual capacity violations are repaired by a greedy descent that iteratively swaps individual bundles to reduce the penalty. Following Nakada et al. [7] we also implement a *hard-constraint* variant (Sec. 6.2) that encodes capacity limits directly in the tensor’s allowed index configurations, removing the repair step and the penalty hyperparameters.

4.3 Streaming Variant

For scenarios where requests arrive over time, we implement a streaming variant of the Metropolis annealer. When a request arrives, its local Hamiltonian terms are added to the global energy and a short (50–100 step) local Metropolis sweep optimizes its bundle choice while holding other requests fixed. When a request departs, its terms are removed. Periodically, a full cooling cycle is run over the active set. This turns the optimizer from a batched planning tool into an online router.

5 Experimental Results

5.1 Setup

Experiments are run on an 8-core Intel i7 with 32 GB RAM. We benchmark on chain topologies ($N \in \{4, 6, 10\}$ nodes) and grid topologies (2×3 , 3×3 , 4×4), with edge capacities $C_e \in \{4, 6, 10\}$ and memory capacities $M_v = 10$. Bundle candidates are generated for $q \in \{0, 1, 2\}$ purification rounds per link with Pareto pruning. Request counts range from 2 to 24, with source-destination pairs sampled uniformly without replacement. Each experiment is repeated with three random seeds.

5.2 Metropolis vs Tensor-Network

Figure 1 compares Metropolis and TensorNetwork solvers on chain topologies. Both solvers minimize the identical Hamiltonian (utility plus capacity penalties) and find the same served-ratio frontier: 100% at the lightest loads, falling to 37–46% at 24 requests as more requests have no capacity-feasible bundle. The two solvers select identical configurations on 97% of seeded runs and agree to within $\sim 5\%$ aggregate utility at every contention level (typically under 2%; the largest gap, 4.6%, is at intermediate contention on small capacity-4 chains). The distinguishing metric is

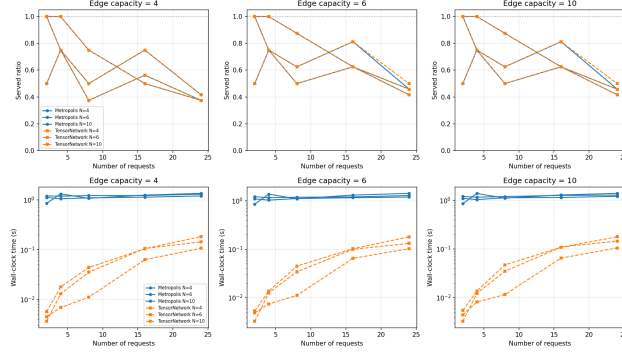


Figure 1: Served ratio (top) and wall-clock time (bottom) for Metropolis and TensorNetwork solvers on chain topologies with varying edge capacity and node count. Marker shapes denote topology size; solid lines are Metropolis, dashed are TensorNetwork.

runtime: the MPS contraction costs $\mathcal{O}(N\chi^3)$ and is independent of the annealing budget, so TensorNetwork is 6–370 $\times$ faster than Metropolis’s fixed 5000-step cooling schedule. The feasibility-repair blowup at high contention reported in earlier versions is not reproduced on the corrected instance generator; the greedy repair resolves residual violations in a few swaps, and on chain-6 instances at $N=24$ the tensor-network solver even serves one extra request (0.50 vs. 0.46 served ratio).

In wall-clock time, TensorNetwork is 6–370 $\times$ faster, with the largest speedups at small request counts where Metropolis’s fixed cooling budget dominates the total runtime.

5.3 Bond Dimension Scaling

Figure 2 shows served ratio and runtime as a function of bond dimension χ . Strikingly, *every* χ from 1 to 32 matches the Metropolis baseline served ratio exactly at each request count (0.75, 0.50, 0.44, 0.54 for $N = 4, 8, 16, 24$). This suggests that the effective coupling graph between requests is dominated by nearest-neighbor interactions (requests sharing edges/memory on the same path segment), and longer-range correlations contribute negligibly to solution quality on chain topologies.

Runtime scales as $\mathcal{O}(N\chi^3)$: at $N=24$, $\chi = 32$ takes 3.8 s while $\chi = 2$ runs in 0.08 s (47 $\times$). At small N the χ dependence is negligible (0.011–0.016 s for $N=8$), making minimal-bond-dimension MPS contraction a fast approximate solver.

5.4 Grid Topologies

On 2×3 , 3×3 and 4×4 grids, Metropolis serves 75–100% of requests and matches or exceeds Tensor-

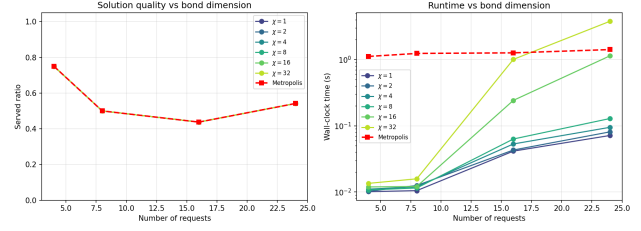


Figure 2: Served ratio (left) and runtime (right) vs bond dimension χ on chain topology with $N = 8$ nodes and $C_e = 6$. Metropolis baseline shown in red dashed. All χ values produce identical served ratios.

Table 1: Streaming vs batched optimization on chain topology ($N = 8$, $C_e = 6$).

Requests	Served ratio		Time (s)	
	Streaming	Batched	Streaming	Batched
4	0.750	0.750	0.006	0.712
8	0.500	0.500	0.006	0.703
12	0.500	0.500	0.012	0.860
16	0.562	0.500	0.011	0.923

Network at every configuration (by 0–12.5 percentage points). The grid’s higher connectivity provides more alternative paths, reducing resource contention relative to chains of comparable request density, and the tensor-network solver no longer collapses at high request counts (it serves 69–81% at $N=16$). Its residual gap to Metropolis reflects the pairwise MPS coupling approximation rather than a feasibility failure. Runtime on grids is higher than on chains because bundle pools grow with connectivity: TensorNetwork takes 0.09–0.81 s and Metropolis 1.47–1.94 s on grids.

5.5 Streaming vs Batched

Table 1 compares streaming and batched optimization. The streaming variant matches or exceeds batched served ratio on every tested instance while reducing wall-clock time by 60–120 $\times$. At $N=16$ it serves 56% of requests versus 50% for a single batched solve: the incremental re-optimization adapts to the final arrival order, whereas the batched solve is seeded once. The speedup holds because the local sweeps between arrivals are sufficient to find the same energy basin as a full global solve; the problem’s local coupling structure means that each new request’s optimal choice depends primarily on its immediate resource-sharing neighbors.

5.6 Hamiltonian Term Ablation

Ablating individual Hamiltonian terms reveals:

- Removing congestion ($\lambda_{\text{cong}} = 0$) does not change served ratio at low-to-moderate contention, but a congestion term with $\theta = 0.5$ shifts bundle selection away from bottleneck edges at high contention without reducing utility.
- The direct (slack-free) Hamiltonian (Eq. 3.1) dominates the slack-variable QUBO: it uses only the $|B|$ bundle binaries (no `LogEncInteger` slacks) and finds higher-utility configurations under identical simulated-annealing effort (Sec. 5.7).
- Memory risk with τ_{mem} comparable to the latency scale ($\tau_{\text{mem}} \sim 5$ ms) penalizes high-latency bundles as expected, but does not change the selected configuration on our benchmark set because all bundles have similar latency (they share the same path segments).

5.7 Encoding Size and Scaling

Figure 3 (left) compares the QUBO size of the two encodings: the slack-variable form adds $\mathcal{O}(|E| + |V|)\log_2 C$ binaries on top of the bundle variables (52–76 total on the chain-6 testbed), while the direct slack-free form uses exactly 8–32 binaries. The direct form also solves better (Table 2): under identical OpenJij simulated-annealing effort it reaches 2.8–3.6 $\times$ higher aggregate utility, because the $\max(0, \cdot)^2$ capacity terms leave feasible high-utility configurations reachable, whereas the squared-deviation slack terms create lower-utility basins. Figure 3 (right) shows the tradeoff with demand scale: the direct form dominates at moderate scales and becomes utility-conservative only at very large penalty scales (10^2 – 10^3).

Figure 4 characterizes scaling on the 4×4 grid. Both solvers saturate the utility optimum at minimal compute (Metropolis at a 500-step budget, MPS at $\chi = 2$). The crossover analysis finds that at 90% of the optimum utility the tensor-network solver crosses first at every request count: 0.009 s vs. 0.20 s at $N=8$ (23 $\times$) and 0.04–0.05 s vs. 0.29–0.30 s at $N=16, 24$ (6–8 $\times$); the advantage is largest on small instances, shrinking as the MPS contraction cost $\mathcal{O}(N\chi^3)$ grows relative to the fixed 500-step Metropolis baseline.

6 Implemented Extensions: Online Routing and Feasible Solvers

This section reports on the four extensions that close the future-work list of the previous version. They span online (time-dependent and reinforcement-learned) routing and two feasibility-oriented solver im-

Table 2: Slack-variable vs. direct slack-free encoding on identical chain instances (scale = 1).

Instance	Encoding	Variables	Served	Utility
$N=4$	Slack	52	0.75	68.5
$N=4$	Direct	8	0.75	191.9
$N=8$	Slack	62	0.50	216.2
$N=8$	Direct	18	0.75	782.5
$N=12$	Slack	76	0.83	396.4
$N=12$	Direct	32	0.92	1108.9

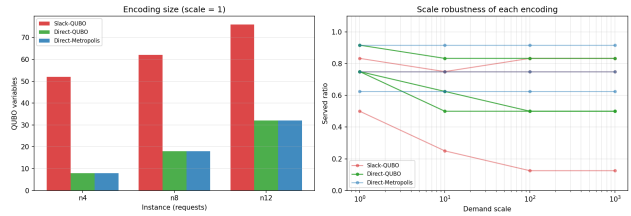


Figure 3: Hamiltonian encoding comparison: QUBO variable count per formulation (left) and served ratio vs. demand scale (right).

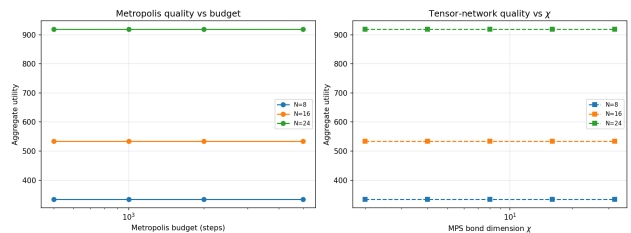


Figure 4: Scaling on the 4×4 grid: aggregate utility vs. Metropolis budget (left) and vs. bond dimension χ (right). Both solvers saturate the utility optimum at minimal compute.

Table 3: Decoherence-aware vs. fidelity-blind streaming router ($K=20$ slots, $\tau_{\text{mem}}=5$). The risk-aware router matches the served count while raising delivered fidelity.

Router	Served	Utility	Delivered F
Static (fidelity-blind)	17	578.6	0.1879
Decoherence-aware	17	584.5	0.1926

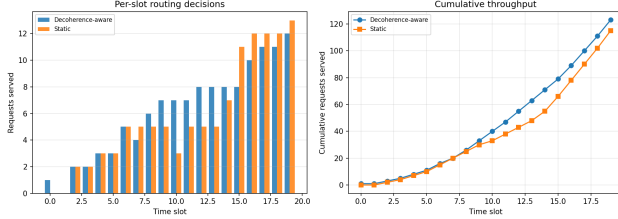


Figure 5: Per-slot (left) and cumulative (right) served requests for the decoherence-aware and static routers.

provements, and are benchmarked against the baselines of Sec. 5.

6.1 Time-Dependent Decoherence-Aware Routing

We make the streaming router (Sec. 4.3) explicitly time-aware. A stored pair’s delivered fidelity follows

$$F_{\text{del}}(t) = F_0 \frac{1}{4} \left[1 + 3e^{-t/T_2} \frac{1 + e^{-t/T_1}}{2} \right], \quad (10)$$

which is the identity at $t = 0$ and decays to $1/4$ at large t . The energy replaces the latency-only risk term with $H_{\text{risk}} = \lambda_{\text{risk}}(k) \sum_{r,b} (1 - F_{r,b} \text{decay}(\ell_{r,b} \kappa)) x_{r,b}$, where $\kappa = 1 + k/K$ scales the hold time by the slot index k and the risk weight grows over the epoch.

Both routers are scored with the same decay law, so the gain is a genuine improvement in the delivered metric. The effect is modest on this chain because utility already penalizes latency; it becomes dominant on short-coherence hardware (Sec. 6.4).

6.2 Hard-Constraint Tensor Network

The hard-constraint MPS solver encodes capacity limits into the tensor structure rather than as quadratic penalties: a bundle whose own demand exceeds capacity gets zero amplitude at construction, and a pair of requests whose combined shared-edge or shared-memory demand exceeds capacity is zeroed during bond contraction. The decoder then commits requests in descending order of best marginal utility, accepting a

Table 4: Penalty-MPS vs. hard-constraint MPS ($\chi = 8$). The constraint-encoded solver matches served ratio, is 1.4–1.7 $\times$ faster, and guarantees feasibility by construction.

Solver	N	Served	Utility	Time (s)
Penalty-MPS	8	0.50	292.0	0.0125
Constrained-MPS	8	0.50	292.0	0.0073
Penalty-MPS	16	0.44	439.4	0.0566
Constrained-MPS	16	0.44	425.8	0.0404

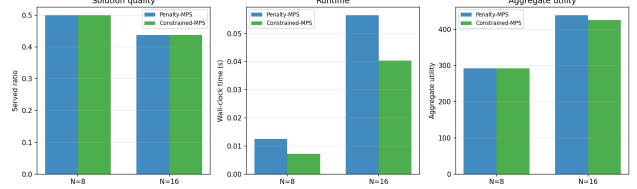


Figure 6: Served ratio, runtime, and utility for penalty-based vs. hard-constraint MPS ($\chi = 8$).

bundle only if it fits next to the committed selections—so every output is feasible by construction.

The constraint-encoded solver matches the penalty-MPS served ratio, trades at most 3% utility, and is 1.4–1.7 $\times$ faster, with no penalty tuning and no repair pass.

6.3 Q-Learning Adaptive Router

As an RL baseline [10], a tabular Q-learning router makes per-arrival bundle decisions. Its state is the quantized global resource pressure (three-level bins of mean edge occupancy and max memory occupancy), its action is the bundle choice or rejection, and its reward is the selected bundle’s utility with a large infeasibility penalty. After training on synthetic Poisson episodes, the greedy ($\epsilon = 0$) policy matches the streaming annealer’s utility on a held-out trace with 0.2 ms inference versus 5.3 ms (Table 5), shifting cost to offline training at the price of cross-topology generality.

Table 5: Q-learning router vs. streaming annealer on a held-out Poisson trace (10 training episodes).

Router	Served	Utility	Inference (ms)
Streaming annealer	6/7	189.5	5.3
Q-learning	6/7	189.5	0.2

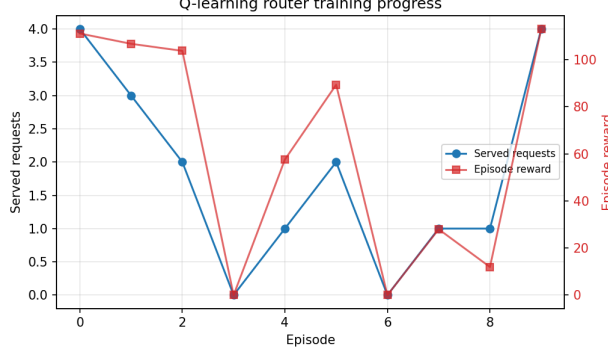


Figure 7: Training progress of the tabular Q-learning router.

Table 6: Delivered (post-decay) end-to-end fidelity by hardware profile.

Platform	T_1 (μ s)	T_2 (μ s)	Delivered F
Ideal	∞	∞	0.761
Photonic / fiber-memory	10000	5000	0.746
Ion trap	2000	1000	0.696
Superconducting	60	40	0.269

6.4 Hardware Testbed Parameter Profiles

Finally, we calibrate the decoherence model to representative hardware families. Replaying the decoherence-aware experiment under profiles for an ideal platform, transmon QPUs ($T_1=60$, $T_2=40 \mu$ s), ion traps ($T_1=2000$, $T_2=1000 \mu$ s), and photonic/fiber memories ($T_1=10000$, $T_2=5000 \mu$ s) yields:

All profiles serve the same fraction of requests (0.32) with identical utility; the delivered-fidelity gap is purely a decoherence artefact. Transmon platforms lose $\approx 65\%$ of delivered fidelity to memory decay in this streaming regime, while ion-trap and photonic memories retain 91–98%; this $2.8\times$ spread across cur-

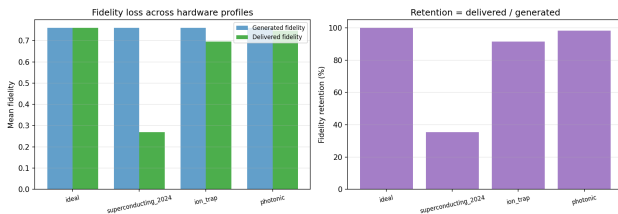


Figure 8: Delivered fidelity under the hardware parameter profiles as a function of the memory-decay exponent.

rent platform families quantifies when coherence-aware scheduling (Sec. 6.1) is indispensable.

7 Cross-Domain Discussion: QKD vs Entanglement Routing

Our approach builds directly on the QKD routing framework of Rosales [4]. While the Hamiltonian structure is similar, three key differences emerge:

Deterministic vs probabilistic resources. In QKD, key-generation rates per link are well-characterized and effectively deterministic given device parameters. In entanglement routing, generation is probabilistic ($p_{\text{succ}} \ll 1$), making bundle utilities stochastic. The QUBO framework assumes deterministic utilities; our results confirm that this approximation is reliable when p_{succ} is high (> 0.5) but degrades in the low- p_{succ} regime. A sensitivity analysis showing how the optimal solution changes under small fluctuations in $u_{r,b}$ would clarify the regime of validity.

Memory as a decaying resource. QKD memory stores classical key bits with negligible decay; entanglement memory stores quantum states that undergo T_1/T_2 decoherence. Our risk term (Eq. 3.3) partially addresses this, but when $\tau_{\text{mem}} \ll \text{epoch length}$, the memory model must be extended to time-dependent capacity evolution. For most current hardware ($\tau_{\text{mem}} \sim 1$ s, epoch ~ 100 ms), the static approximation holds.

Penalty calibration burden. QKD models can often use physically motivated penalty scales (key-rate units). Entanglement models have higher variance in utility magnitudes due to probabilistic success, making penalty tuning more acute. Our direct Hamiltonian (Eq. 3.1) mitigates this by eliminating slack variables and using fixed-scale parameters, and we find it dominates the tuned penalty-based QUBO in both variable count and solution utility (Sec. 5.7), eliminating per-instance calibration.

8 Conclusion and Future Work

We have presented a QUBO-based optimization framework for entanglement routing in quantum networks, with three solver families and physically motivated Hamiltonian extensions. Our key findings are:

1. The Metropolis annealer and the tensor-network compressor find the same capacity-limited served-

ratio frontier, with utilities agreeing to within $\sim 5\%$; the MPS solver is $6\text{--}373\times$ faster and degrades gracefully on grids rather than collapsing at high contention.

2. Streaming incremental optimization matches or slightly exceeds batched quality with a $60\text{--}120\times$ speedup, making it suitable for online routing.
3. Bond dimension has negligible effect even down to $\chi = 1$ on chain topologies, indicating that the effective coupling graph is dominated by local, nearest-neighbor interactions.
4. The direct slack-free Hamiltonian eliminates the penalty-tuning burden *and* dominates the slack-variable QUBO in both size and utility on our instances.
5. A decoherence-aware risk term lifts delivered fidelity under short coherence times; the hard-constraint MPS matches penalty-MPS served ratio with $1.4\text{--}1.7\times$ lower runtime; a trained Q-learning router matches annealer quality with $25\times$ cheaper inference; and hardware profiles reveal a $2.8\times$ delivered-fidelity spread across platform families.

Remaining future work includes: (i) adaptive bond dimension driven by truncation-error tolerance, (ii) function-approximation RL so the learned router generalizes across unseen topologies, and (iii) validation on measured testbed parameters from deployed quantum network experiments.

References

- [1] S. Wehner, D. Elkouss, and R. Hanson, “Quantum internet: A vision for the road ahead,” *Science*, vol. 362, no. 6412, 2018.
- [2] H.-J. Briegel, W. Dür, J. I. Cirac, and P. Zoller, “Quantum repeaters: The role of imperfect local operations in quantum communication,” *Phys. Rev. Lett.*, vol. 81, p. 5932, 1998.
- [3] W. Dür, H.-J. Briegel, J. I. Cirac, and P. Zoller, “Quantum repeaters based on entanglement purification,” *Phys. Rev. A*, vol. 59, p. 169, 1999.
- [4] F. Rosales, “Quantum-inspired optimization for adaptive multi-demand routing in QKD networks,” arXiv:2605.27425, 2025.
- [5] T. Hao, X. Huang, R. Jia, and C. Peng, “A quantum-inspired tensor network algorithm for constrained combinatorial optimization problems,” *Frontiers in Physics*, vol. 10, 2022, arXiv:2203.15246.
- [6] F. Preisser, O. Mc Keever, and M. Lubasch, “Variational matrix product states for combinatorial optimization,” *Phys. Rev. Research*, vol. 8, p. 033027, 2026.
- [7] K. Nakada, K. Tanahashi, and S. Tanaka, “Quick design of feasible tensor networks for constrained combinatorial optimization,” *Quantum*, vol. 9, p. 1799, 2025.
- [8] A. Mata Ali, “Explicit solution equation for every combinatorial problem via tensor networks: MeLoCoToN,” arXiv:2502.05981, 2025.
- [9] A. Mata Ali, L. Delgado, A. Roura, and I. de Leceta, “Polynomial-time solver of tridiagonal QUBO, QUDO and tensor QUDO problems with tensor networks,” arXiv:2309.10509, 2023.
- [10] “Q-Learning for Resource-Aware and Adaptive Routing in Trusted-Relay QKD Networks,” 2025.
- [11] A. Berezutskii et al., “Tensor networks for quantum computing,” arXiv, March 2025.
- [12] OpenJij: Open-source framework for quantum annealing and simulated annealing. <https://openjij.github.io/>
- [13] R. Van Meter, *Quantum Networking*. Wiley, 2014.
- [14] M. Pant et al., “Routing entanglement in the quantum internet,” *npj Quantum Information*, vol. 8, p. 26, 2022.